

QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

Group Name: Last minute

Group Leader: Liew Jian Wei

Table of Content

Document Control

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability
2. Risk Assessment & Mitigation Strategy
3. Test Environment & Execution Strategy
4. CI/CD Release Thresholds & Automation Gates
5. Test Case Specifications (Drafts)
6. Test Strategy & Plan Sign-Off

Document Control

Field	Detail
System Under Test (SUT)	StitchFlow V2 – AI-powered HITL Document Workflow Engine
Team Repo URL	https://github.com/Kisryo/StitchFlow#
Project Board URL	Not applicable. Task planning and tracking were maintained using Markdown-based documentation (Navigation.md, overview.md, etc...) within the VS Code environment and version-controlled via GitHub repository.
Live Deployment URL (optional for preliminary round)	Not publicly deployed. Frontend runs locally at http://localhost:5173/ during development.

Objective:

The primary objective of this Quality Assurance Testing Documentation (QATD) is to ensure that **StitchFlow V2** reliably processes documents through its AI-powered workflow pipeline, including ingestion, redaction, reasoning, and policy screening while maintaining system stability, correctness and performance under various scenarios.

The system must:

- Accurately process and extract document content (PDF, DOCX, TXT, Email)
- Ensure PII is correctly detected and redacted
- Generate reliable AI-driven reasoning outputs using GLM-4.5
- Enforce policy compliance with minimal false positives/negatives
- Execute LangGraph workflow state transitions correctly
- Support human-in-the-loop interruptions and resumption
- Maintain audit logs for all decisions
- Handle failures properly without system crashes
- Ensure frontend-backend consistency

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

This section defines how each feature is linked back to system requirements, ensuring that every test case and implementation step is traceable to a specific functional requirement.

- Every feature (e.g., PII Redaction, Policy Screening, Task Orchestration) is mapped to a numbered requirement.
- Testing is not random — each test case validates a specific requirement or user story.
- Helps to prevent missing functionality during development.
- Ensures full coverage between: Requirement → Implementation → Testing → Validation

1.1 In-Scope Core Features

✓ Document Upload & Ingestion

- Handles file submission and extracts raw text from supported formats (PDF, DOCX, TXT, Email).
- Ensures documents are structured and ready for downstream AI processing.

✓ PII Redaction Engine

- Detect sensitive data (emails, phone numbers, names)
- Replace with masked tokens
- Maintain reversible mapping

✓ GLM AI Reasoning Chain

- Classify document type
- Extract structured entities
- Identify ambiguities/conflicts
- Generate recommendations

✓ **Policy Screening Engine**

- Match recommendations against compliance rules
- Assign risk scores
- Trigger review if violations detected

✓ **Human-in-the-loop system**

- Pauses workflow execution when clarification or approval is required.
- Allows human intervention to validate or modify AI-generated decisions.

✓ **LangGraph Workflow Engine + Task Orchestration**

- End-to-end orchestration of document workflow states using LangGraph state machine
- Automates full pipeline: ingestion → redaction → reasoning → policy screening → decision execution
- Supports human-in-the-loop interruptions for clarification and policy approval
- Optional external task execution:
 - Google Sheets sync (data logging / structured export)
 - Gmail draft creation (automated email generation from recommendations)

✓ **Audit Logging System**

- Implements append-only audit trail for every workflow action
- Logs ALL critical system events:
 - Workflow state transitions
 - AI reasoning inputs & outputs (GLM calls)
 - Policy screening decisions & risk scores
 - Human approvals / rejections / clarifications
- Generates tamper-proof log entries using hashing

✓ **Dashboard & Workflow Tracking**

- Provides real-time visualization of workflow states and processing stages.
- Enables monitoring of AI outputs, policy violations and approval actions.

1.2 Out-of-Scope

- **Production deployment scaling**

- The system does not currently include infrastructure-level scaling for high-availability production environments.

- Deployment is limited to local or single-instance execution without load balancing, auto-scaling or distributed cloud architecture.

- **Multi-user concurrency optimization**

- The system does not support advanced concurrency control for multiple simultaneous users beyond basic API handling.

- Features such as session isolation, conflict resolution and concurrent workflow locking are not implemented.

- **Advanced authentication/authorization**

- The system does not include enterprise-grade security mechanisms such as role-based access control (RBAC), OAuth2 or multi-factor authentication.

- User access control is minimal and primarily intended for development and testing purposes.

- **Real-time collaboration features**

- The system does not support live multi-user collaboration on workflows or shared document editing.

- Features such as real-time synchronization, collaborative decision-making or shared dashboards are excluded from the current scope.

2. Risk Assessment & Mitigation Strategy

This section identifies potential technical and operational risks within the StitchFlow V2 system. Each risk is evaluated based on likelihood and severity, followed by corresponding mitigation strategies and validation/testing approaches.

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
AI hallucination in reasoning <i>(Eg. The AI model may generate inaccurate or fabricated outputs when handling ambiguous and complex inputs.)</i>	4	5	20 (Critical)	Use structured prompts + schema (Pydantic) validation + enforce human review for low-confidence outputs.	Use adversarial and ambiguous documents to evaluate consistency, accuracy and confidence scoring.
PII not fully detected <i>(Eg. Sensitive information may be missed due to limitations in pattern matching or diverse data formats.)</i>	3	5	15 (High)	Enhance regex patterns + add NLP-based fallback detection + validate token mapping.	Test with real PII samples (eg. datasets containing emails, phone numbers, mixed format) and verify redaction accuracy.
Backend crash during upload <i>(Eg. Invalid, corrupted, or oversized files may cause the backend service to fail during processing.)</i>	3	4	12 (High)	Enforce file validation (type, size) + implement exception handling and safe failure responses.	Upload corrupted, unsupported, and oversized files to verify system stability.
Frontend/backend desync <i>(Eg. Inconsistent data states may occur when updates between frontend and backend are not properly synchronized.)</i>	3	4	12 (High)	Implement API sync validation + idempotent requests + controlled update handling.	Simulate concurrent updates and rapid API calls to verify synchronization consistency.

Large file crash <i>(Eg. Processing high-size documents may exceed system memory or processing limits, leading to failures.</i>	4	3	12 (High)	Set strict file size limits + optimize processing pipeline and memory handling.	Set strict file size limits + optimize processing pipeline and memory handling.
API failure (GLM downtime) -Eg. External AI service unavailability may interrupt reasoning and halt workflow progression.	3	4	12 (High)	Implement retry logic + timeout handling + fallback workflow behavior.	Simulate API timeouts and failures to ensure graceful degradation.
API connection failure (frontend ↔ backend) -Eg. Communication breakdown between frontend and backend may prevent user interactions and data retrieval.	3	3	9 (Medium)	Add retry mechanism + clear error handling UI + loading/fallback states.	Simulate backend offline scenarios and network interruptions.
Workflow state corruption (Eg. Invalid or inconsistent state transitions may disrupt the workflow execution and data integrity.	3	5	15 (High)	Enforce LangGraph state machine validation + persist state after each step.	Inject forced failures and invalid transitions to verify state integrity. (e.g., skip from <i>New</i> → <i>Parsed</i>)

<i>Likelihood (1-5)</i>		<i>Severity (1-5)</i>	
1	<i>Rare</i>	1	<i>Impact is Negligible</i>
2	<i>Unlikely</i>	2	<i>Impact is Minor</i>
3	<i>Possible</i>	3	<i>Moderate Impact</i>
4	<i>Likely</i>	4	<i>Major Impact</i>
5	<i>Almost Certain</i>	5	<i>Critical Failure of the system</i>

Risk Score = Likelihood × Severity

Risk Level Reference:

<i>Risk Score</i>	<i>Risk Level</i>	<i>Recommended Action</i>
1 – 5	<i>Low</i>	<i>Monitor only. Acceptable risks.</i>
6 – 10	<i>Medium</i>	<i>Mitigate + Testing</i>
11 – 15	<i>High</i>	<i>Must need mitigating and through testing is required</i>
16 – 25	<i>Critical</i>	<i>Priority is Highest. Need extensive level of testing.</i>

3. Test Environment & Execution Strategy

The testing of StitchFlow V2 is conducted in a **local development environment** to ensure controlled, consistent and reproducible results across different test scenarios.

- The backend is deployed using **FastAPI (localhost:8000)** while the frontend runs on **React (localhost:5173)**, enabling real-time interaction between user interface and API services.
- The system uses **SQLite** as the primary database for lightweight testing and rapid setup, allowing quick data storage, retrieval and reset without requiring complex configuration.
- All components (frontend, backend, database) are executed within the **VS Code environment**, providing an integrated workspace for development, debugging and testing.
- This setup closely simulates actual system behavior while maintaining simplicity, making it suitable for functional, integration and regression testing.

Unit Testing

- **Scope:** Validation of individual components such as document ingestion logic, PII detection engine, GLM reasoning functions, and policy screening rules
- **Execution:** Implemented using *pytest*. Tests are executed during development and automatically triggered in CI pipeline on code commits and pull requests
- **Isolation:** External dependencies such as GLM API, database, and file storage are mocked to ensure focus on core logic only
- **Pass Condition:** Correct handling of:
 - Valid inputs (*happy path scenarios*)
 - Invalid inputs (e.g., empty files, malformed text)
 - Edge cases (e.g., ambiguous or mixed-format PII)
Expected outputs must match defined response formats (e.g., structured redaction output or error handling messages)

****Happy Path: Document Processing Workflow****

1. User uploads a valid PDF file
2. System successfully extracts text
3. PII is correctly detected and redacted
4. GLM generates reasoning output without API errors
5. Policy screening runs successfully
6. Workflow reaches “ReadyForReview” state

Integration Testing

- **Scope:** End-to-end workflow validation across modules (Document Upload → PII Redaction → AI Reasoning → Policy Screening → Human-in-the-loop)
- **Execution:** Performed after unit testing completion. Real API calls and database interactions are used without mocking to validate full system integration
- **Workflow:** Ensures data flows correctly between services such as FastAPI backend, LangGraph workflow engine, GLM API, and SQLite database
- **Pass Condition:** Integrated components must:
 - Execute full workflow without failure
 - Produce correct state transitions (e.g., Ingested → Redacted → Parsed → Reviewed)
 - Return expected final outputs (recommendations, risk scores, and review states)

Test Environment (CI/CD Practice)

- **Local Testing:** Manual testing performed on localhost environment using VS Code (frontend: <http://localhost:5173>, backend: <http://localhost:8000>)
- **CI Pipeline (Staging):** GitHub Actions triggers automated tests on every push and pull request
- **Automated Testing Flow:**
 - Unit tests executed first (fast validation of components)
 - Integration tests executed on PR merge attempts to main branch
 - Failures block deployment to ensure stability before release

Regression Testing & Pass/Fail Rules

- **Execution Phase:** Conducted whenever new features or updates are merged into the main branch
- **Scope Coverage:** Ensures all core modules (ingestion, reasoning, policy screening, HITL workflow) remain functional after changes
- **Pass/Fail Condition:**
 - Test is marked **PASS only if actual output matches expected output exactly**
 - Any deviation (logic failure, state mismatch, API error) is recorded in defect log and marked as FAIL
- **Continuation Rule:** Regression suite must achieve minimum threshold before proceeding to full system validation to ensure baseline stability

Test Data Management & Execution Strategy

Test data is carefully designed to reflect **real-world usage scenarios, system stress conditions and edge-case behaviors** to ensure robustness of the StitchFlow V2 pipeline.

- **Manual datasets:**

Sample real-world-like documents such as:

- Business contracts (e.g., “Service Agreement between Vendor A and Client B”)
 - Event request forms (e.g., “University event budget approval request”)
 - Structured text inputs (e.g., JSON-like or form-based data)
- These simulate actual user submissions and workflow inputs.

- **Automated Data:**

Generated datasets using scripts (SQL seed data or mock generators) for workflows, policy rules and reasoning inputs

- **Generated test data:**

Artificially created files in multiple formats to test system boundaries:

- PDF: multi-page reports with structured sections
- DOCX: formal letters or policy documents
- TXT: unstructured raw text inputs
- Large files (10–50MB): to test upload limits and performance
- Corrupted files: broken encoding or unreadable formats to test failure handling

- **Edge cases:**

Inputs designed to challenge system reliability and AI reasoning:

- Empty files (no content) → tests ingestion failure handling
- Ambiguous text (e.g., “Approve the request soon”) → tests clarification triggering
- Mixed-format PII (e.g., email + phone + partial names in same document) → tests redaction accuracy
- Conflicting information (e.g., different budgets in same document) → tests reasoning consistency

Passing Rate Threshold

- **Overall Requirement:** Minimum **85% of all executed test cases must pass**
- **Critical Test Cases:** Must achieve **100% pass rate**, including:
 - PII redaction accuracy
 - Workflow execution integrity
 - Policy screening correctness
- **Performance Constraint:** API response time should remain within acceptable limit (e.g., <800ms)
- **Acceptance Rule:** System is considered production-ready only when all critical tests pass and overall threshold is satisfied

4. CI/CD Release Thresholds & Automation Gates

This section defines the quality metrics and minimum success criteria that must be satisfied before code is allowed to move to the next stage of the CI/CD pipeline (e.g., from development → staging → production).

HOW IT WORKS?

Each time code is pushed or a pull request is made:

1. Automated tests are triggered (unit, integration, regression)
2. Metrics are calculated (e.g., accuracy, response time, coverage)
3. Results are compared against defined thresholds
4. The pipeline makes a decision:
 - Pass → proceed to next stage (deployment or staging)
 - Fail → block pipeline and require fixes

4.1 Integration Thresholds (Merging to Main)

The integration threshold ensures that the main branch remains a stable and reliable source of truth for StitchFlow V2. At this point, all automated validations such as **build success, unit test completion, linting checks and basic coverage evaluation** must pass before allowing a merge. This prevents unstable or incomplete features from affecting the main system and ensures consistent reliability across development iterations.

Checks	Requirements	Project Result	Pass/Failed
<i>Automatic Build</i>	<i>Zero Build Error</i>	<i>Build has 0 errors</i>	<i>Passed</i>
<i>Unit Tests</i>	<i>100% Passing Rate</i>	<i>100% passed</i>	<i>Passed</i>
<i>Code Quality</i>	<i>Zero Linting Error</i>	<i>97% compliance</i>	<i>Failed</i>
<i>Test Coverage</i>	<i>Minimum 80%</i>	<i>78–79% coverage</i>	<i>Failed</i>
<i>AI Reasoning Accuracy</i>	<i>≥ 80% accuracy</i>	<i>~75% accuracy</i>	<i>Failed</i>
<i>Critical Test Cases</i>	<i>100% must pass</i>	<i>1 critical failure</i>	<i>Failed</i>

4.2 Deployment Thresholds (Pushing to Production)

The deployment threshold ensures that the full end-to-end StitchFlow V2 system is production-ready, stable and performs reliably under real-world conditions before release.

Checks	Requirements	Project	Pass/Failed
<i>Regression Test</i>	<i>≥ 90% pass rate</i>	<i>~92%</i>	<i>Passed</i>
<i>AI Output Pass Rate</i>	<i>≥ 80% correct AI responses across test prompts</i>	<i>~75% accuracy</i>	<i>Failed</i>
<i>Critical Bugs (P0/P1)</i>	<i>Zero critical bugs allowed</i>	<i>1–2 detected</i>	<i>Failed</i>
<i>API Performance</i>	<i>< 800ms response time</i>	<i>~450–900ms (variable)</i>	<i>Mixed</i>
<i>Security Validation</i>	<i>No API key / data leakage</i>	<i>Clean</i>	<i>Passed</i>

5. Test Case Specifications (Drafts)

This section defines the core test scenarios used to validate StitchFlow V2’s end-to-end workflow, including document ingestion, AI reasoning, PII redaction, and policy screening.

Test cases are designed to ensure both **functional correctness** (system behaves as expected) and **non-functional reliability** (performance, stability, and robustness under load).

Each test case is categorized into:

- **Happy Path (End-to-End Flow)** – validates complete system workflow
- **Negative / Edge Case** – verifies error handling and system resilience
- **Non-Functional Tests (NFR)** – evaluates performance and system efficiency under load

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-01	Happy Case (Full Workflow) – Document Processing Pipeline	Validate complete workflow from document upload to AI reasoning and policy screening	1. Upload a valid PDF contract document 2. Trigger ingestion pipeline 3. System extracts text successfully 4. PII (emails, phone numbers) is detected and masked 5. GLM generates document classification and insights 6. Policy engine evaluates compliance rules 7. Workflow completes and the output appears in dashboard	Document should be processed successfully across all stages. PII redacted correctly, AI output generated and policy risk score displayed	Pipeline completes end-to-end with correct outputs across all modules. All stages executed without failure. Status: Passed

TC-02	Specific Case (Edge): Invalid Document Upload Handling	Tests system robustness when unsupported or corrupted files are uploaded, ensuring backend stability and proper error handling	1. Upload unsupported file type (e.g., .exe or corrupted PDF) 2. Attempt ingestion into pipeline 3. Trigger document processing workflow	System rejects file gracefully with clear UI error message; no backend crash; workflow not initiated.	System does not process file but error feedback is partially unclear (no structured message returned) - throws backend error instead of controlled validation message. Status: Failed
TC-03	NFR (Performance): AI Reasoning Response Time	Validate GLM reasoning API performance under normal load	1. Upload standard document 2. Trigger reasoning module 3. Measure response time	AI reasoning should complete within acceptable threshold (<800ms ideal target)	Response time fluctuates between 500–900ms Status: Mixed
TC-04	NFR (Stability) – Policy Screening Under Load	Test policy engine stability when multiple documents are processed	1. Process multiple documents sequentially 2. Trigger policy screening repeatedly	System should maintain stable execution without delay spikes or failures	System remains stable but slight latency observed under repeated calls Status: Passed

6. AI Output & Boundary Testing (Drafts)

This section validates the behaviour of the GLM-based AI reasoning module in StitchFlow V2. It ensures that the model produces **accurate, structured and policy-aligned outputs** while also handling abnormal, ambiguous or adversarial inputs without causing system instability. The focus is on both **output correctness** and **robust failure handling** within the document processing pipeline.

6.1 Prompt / Response Test Pairs

This subsection defines a set of controlled prompt–response scenarios used to evaluate the GLM reasoning engine. Each test pair includes a structured input prompt and an expected output behaviour, along with clear acceptance criteria to determine whether the AI response is valid, partially acceptable or failing.

The goal is to ensure that the model consistently:

- Extracts correct information from documents
- Maintains structured reasoning output
- Handles ambiguity without hallucination
- Aligns outputs with system policy constraints

Each test case is evaluated against defined **correctness, completeness and safety criteria** to ensure reliability in real-world document processing scenarios.

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	Extract and summarize key obligations from a legal contract document provided in text form	Output should correctly identify key obligations, be structured in bullet points or short paragraphs, and must not hallucinate clauses not present in the input document.	AI generates structured summary with correct obligations extracted from document.	Pass
AI-02	Identify and redact all PII (emails, phone numbers, names) from a mixed-format document	All PII must be correctly detected and replaced with masked tokens (e.g. [REDACTED_EMAIL]) No sensitive data should remain visible.	Partial redaction performed; some phone number formats not detected	Fail

AI-03	Classify a document and provide compliance risk score using policy rules	Output must include correct document classification (e.g. contract, request), a valid risk score (0–100) and a justification aligned with policy engine rules.	Classification is correct but justification is partially generic and lacks policy mapping detail.	Partial Pass
AI-04	Detect ambiguity or conflicting information in a user-submitted incident report document.	Output must clearly flag ambiguous or conflicting statements, explain why they are uncertain, and trigger a “requires human review” recommendation in HITL workflow.	Model identifies some ambiguity but does not clearly label conflict severity or trigger review flag properly.	Partial Pass
AI-05	Process an adversarial or unstructured text input (random mixed sentences with no clear format) and extract meaningful entities if available	System should not crash; must return either “insufficient structured data” or extract only valid entities without hallucinating missing information.	Model attempts weak extraction and introduces a few inferred (non-existent) entities.	Fail
AI-06	Given a structured incident report document, extract key entities (people, dates, events) and map them into a structured JSON format.	Output must correctly extract entities with accurate labels, follow strict JSON structure, contain no extra or hallucinated fields, and maintain consistency with input text.	Entities extracted correctly but JSON structure includes one extra inferred field not present in source.	Partial Pass

6.2. Oversized/Larger Input Test

This section evaluates how StitchFlow V2 handles **inputs that exceed the system’s defined processing limits**, particularly in document ingestion and AI reasoning modules. It ensures the system remains stable and does not crash or produce unpredictable outputs when users upload excessively large or complex documents.

The test focuses on verifying whether the system correctly **enforces input constraints**, applies safe handling mechanisms (e.g. rejection or chunking) and maintains consistent behaviour under load.

Fields	Details
Maximum Input Size	10MB file size limit (approx. 3–5 pages of structured text per document depending on format)
Input used while testing	Large document (~20MB / multi-page contract with repeated sections and embedded text data)
Expected Behavior	System should reject input with a clear validation message or safely chunk document into smaller segments for processing without crashing.
Actual Behavior	System processes part of the document but becomes unstable during later processing stages (partial ingestion observed).
Status	Partial pass

6.3. Adversarial/Edge Prompt Test

This section evaluates how StitchFlow V2 handles **uncertain, ambiguous or low-structure inputs** that are commonly found in real-world user submissions. These inputs are intentionally designed to test the robustness of the GLM reasoning engine, especially its ability to avoid hallucination, detect missing information, and trigger Human-in-the-Loop (HITL) escalation when required.

The expected system behaviour is not perfect extraction, but rather **controlled reasoning, uncertainty detection and safe fallback handling**.

Field	Details
Test ID	AI-06
Prompt Input	“Need to book a room maybe around mid next month afternoon time, around 20–30 people. Not sure about budget maybe a few thousand. Contact later, Sarah.”
Expected Behavior (Acceptance Criteria)	System should identify ambiguous and incomplete fields (e.g., vague date, time range, unclear budget, and tentative intent). Only high-confidence entities should be extracted. System must avoid hallucinating exact values and should trigger clarification via HITL when uncertainty is high.
Actual System Output (Post-Improvement)	The system successfully performed structured extraction and ambiguity detection. It identified partial entities such as <i>room booking request</i> , <i>approximate date (15th of next month)</i> , <i>afternoon time</i> , and <i>attendee range</i> , while explicitly flagging unclear or missing information. The system also generated clarification questions for each ambiguous field and avoided forcing precise values.
System Behaviour Summary	The workflow correctly classified the request as an inquiry and triggered ambiguity detection across multiple fields (date, time, attendees, intent, and budget). It generated structured entities with confidence scores and highlighted 8 ambiguity cases, ensuring no hallucinated values were introduced. HITL escalation was effectively implied through recommendation outputs.
Status	Pass

6.4. Hallucinating Handling

This section defines how StitchFlow V2 detects and mitigates **AI hallucination**, where the GLM model may generate information that is not supported by the input document or system data.

Hallucination control is critical in the system because StitchFlow V2 is used for **document processing, compliance checks and decision support** where incorrect AI outputs can lead to invalid reasoning or policy violations.

Hallucination handling is implemented through multiple layers of control:

- **Structured Prompting:** The GLM is constrained to produce outputs in predefined formats (e.g., JSON or structured fields), reducing free-form generation errors.
- **Input Grounding:** Model responses are required to be based strictly on extracted document content, avoiding external assumptions.
- **Output Validation:** Generated results are checked against expected schema fields (e.g., missing or extra fields are flagged).
- **Confidence Filtering:** Low-confidence or uncertain outputs are identified and marked for review.
- **Human-in-the-Loop (HITL):** When hallucination risk is detected, the workflow is paused and forwarded for human verification instead of auto-processing.
- **Audit Logging:** All AI outputs are recorded to support traceability and post-analysis of incorrect reasoning patterns.

To ensure continuous improvement, these mechanisms are refined iteratively throughout development following an **Agile development approach**, where hallucination test cases are repeatedly evaluated and system behaviour is adjusted across development cycles.

****Agile Iterative Testing Note****

In alignment with Agile development principles, hallucination detection and mitigation are not tested only at the final stage. Instead, they are continuously validated throughout development iterations, allowing improvements to:

- Prompt design
- Model behaviour tuning
- Validation logic refinement